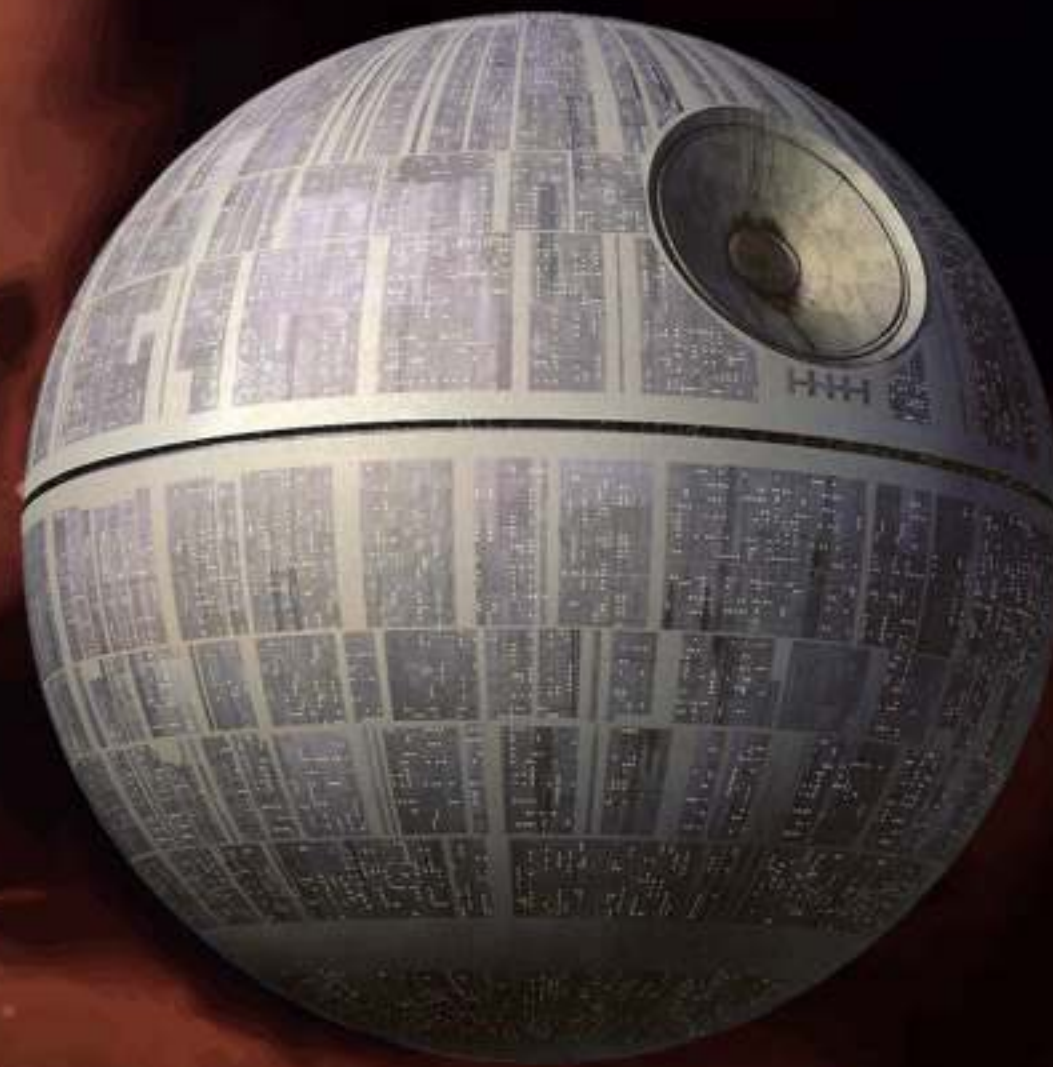


# Les pools de thread (épisode II)



**Fabrice.Kordon@lip6.fr**





# Callable versus Runnable

2

## Runnable

-  `void run()`
-  Impossible de récupérer une valeur de retour

## Callable<V>

-  `V call()`
-  Récupération d'une valeur à l'issue de l'exécution



**Deux mécanismes...**

Complémentaires



# L'interface Future<V>

3

## Expression d'un résultat asynchrone

## Méthodes

-  **boolean cancel (boolean mayInterruptIfRunning)**
  - ▶ Annuler l'exécution de la tâche avant produire le résultat
-  **V get()**
  - ▶ Récupère la valeur associée a calcul
  - ▶ Attente du calcul si nécessaire
-  **V get(long timeout, TimeUnit unit)**
  - ▶ Attendre un certain temps la valeur associée au calcul
  - ▶ TimeoutException levée en cas de dépassement de la temporisation
  - ▶ Peut aussi lever CancellationException, ExecutionException, InterruptedException
-  **boolean isCancelled()**
-  **boolean isDone()**



# Nouvelles méthodes dans l'interface ExecutorService

4

## Gestion des threads «callable»

 Lancer l'exécution d'une tâche

▶ `<T> Future<T> submit(Callable<T> task)`

 Exécuter toutes les tâches et récupérer une liste de résultats

▶ `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)`

▶ `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`

 Exécuter une tâche et récupérer le résultat associé

▶ `<T> T invokeAny(Collection<? extends Callable<T>> tasks)`

▶ `<T> T invokeAny(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`



**RTFM!**



# Nouvelles méthodes dans l'interface ExecutorService

4



## Gestion des threads / collection



### Résultat indéfini si...

La collection de tâches est modifiée après l'appel à invokeAll

• Lancer l'exécution

▶ `<T> Future<T>`

• Exécuter toutes les tâches et récupérer une liste de résultats

- ▶ `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)`
- ▶ `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`

• Exécuter une tâche et récupérer le résultat associé

- ▶ `<T> T invokeAny(Collection<? extends Callable<T>> tasks)`
- ▶ `<T> T invokeAny(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`

**RTFM!**





# En guise de conclusion...

5

## Il est possible désormais

- De créer des threads «sans retour»
- De lancer des calculs parallèles qui rendent des valeurs
  - ▶ Proche de la notion de «promesses»
  - ▶ Aussi dans javascript

## En dehors de Java?

- Mécanismes à base de messages (réparti)
  - ▶ Déclencher des actions sur réception
- Super-calculateurs ou GPU
  - ▶ Dans les unités vectorielles

  
**Bientôt des exemples...**

...et en TD/TME, le calcul matriciel